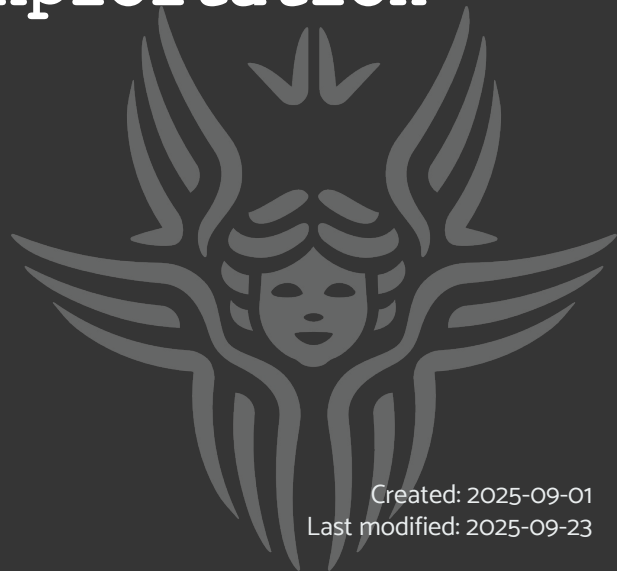
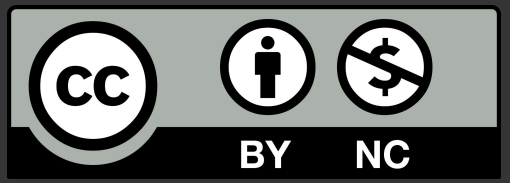
ETHL – Ethical Hacking Lab

0x09 - Intro to glibc Heap Exploitation

Davide Guerri - davide[.]guerri AT uniroma1[.]it
Ethical Hacking Lab
Sapienza University of Rome - Department of Computer Science



Created: 2025-09-01
Last modified: 2025-09-23



ToC

Glibc Dynamic Memory Allocation

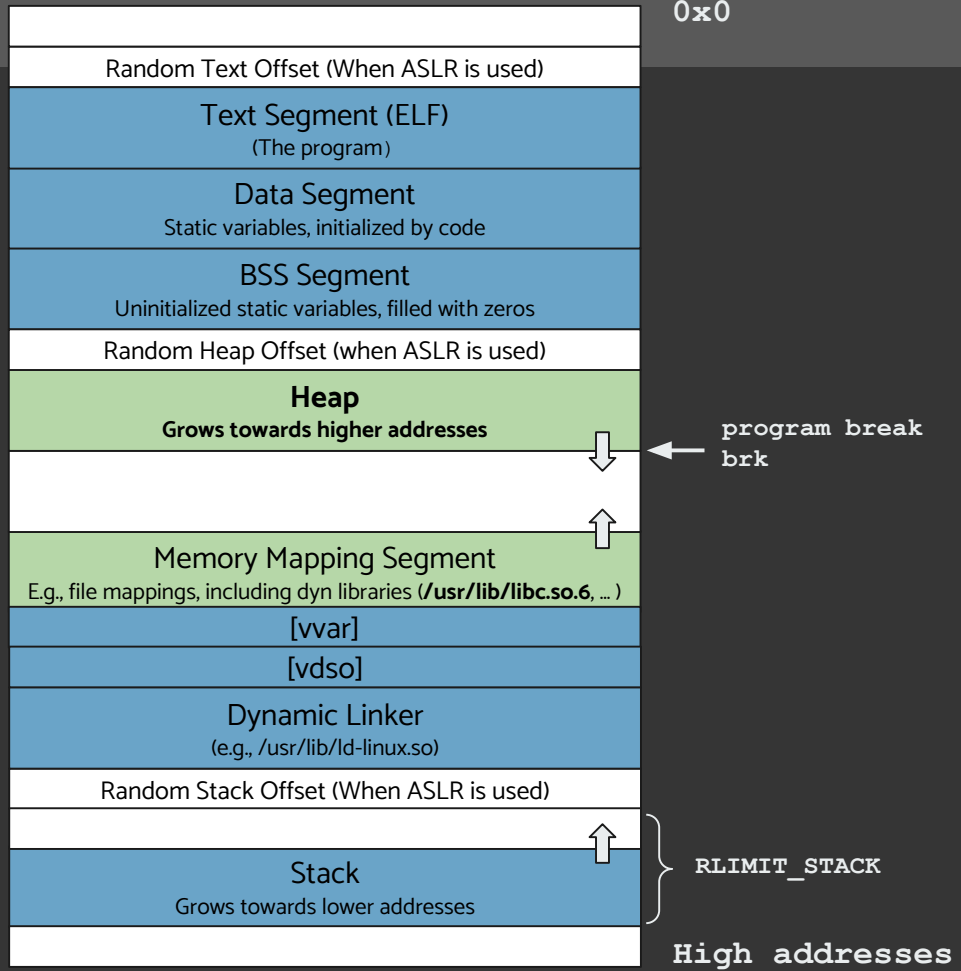
Process virtual memory layout (64 bits arm)

vdso

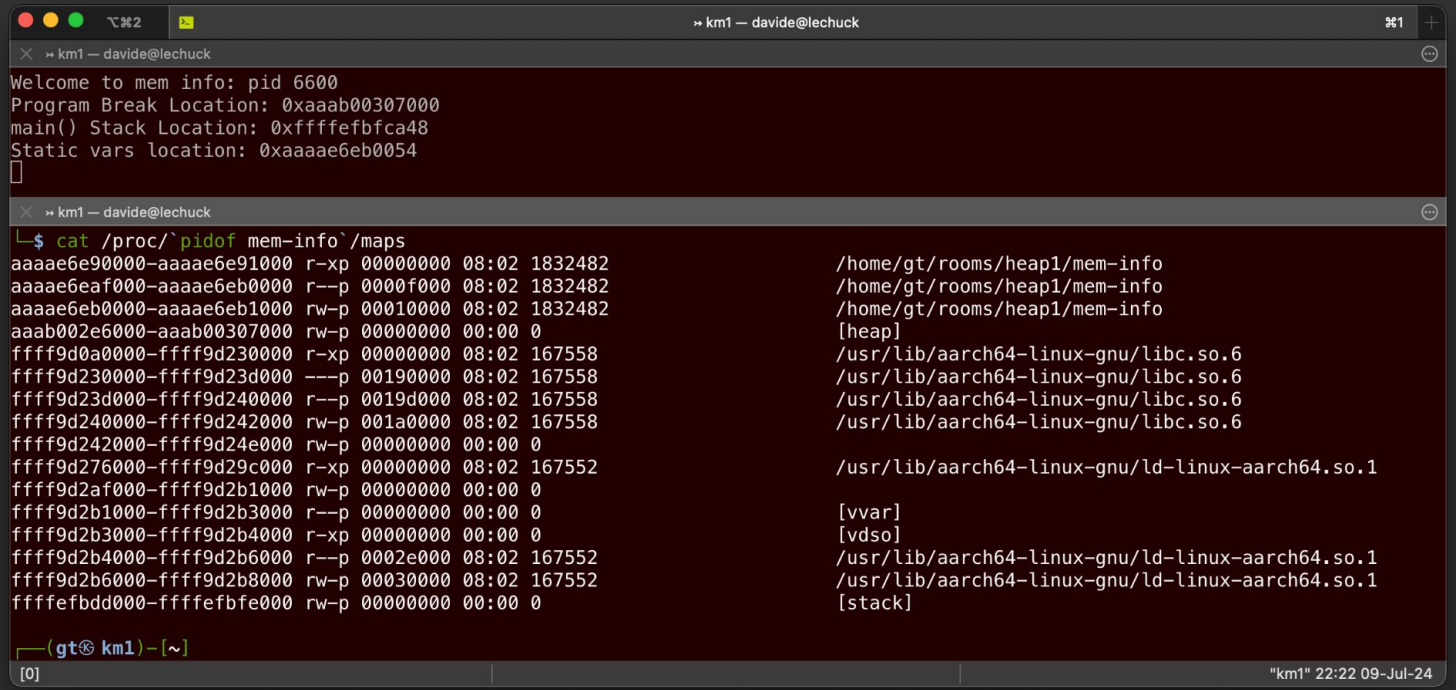
- memory area allocated in user-space which exposes some kernel functionalities

vvar

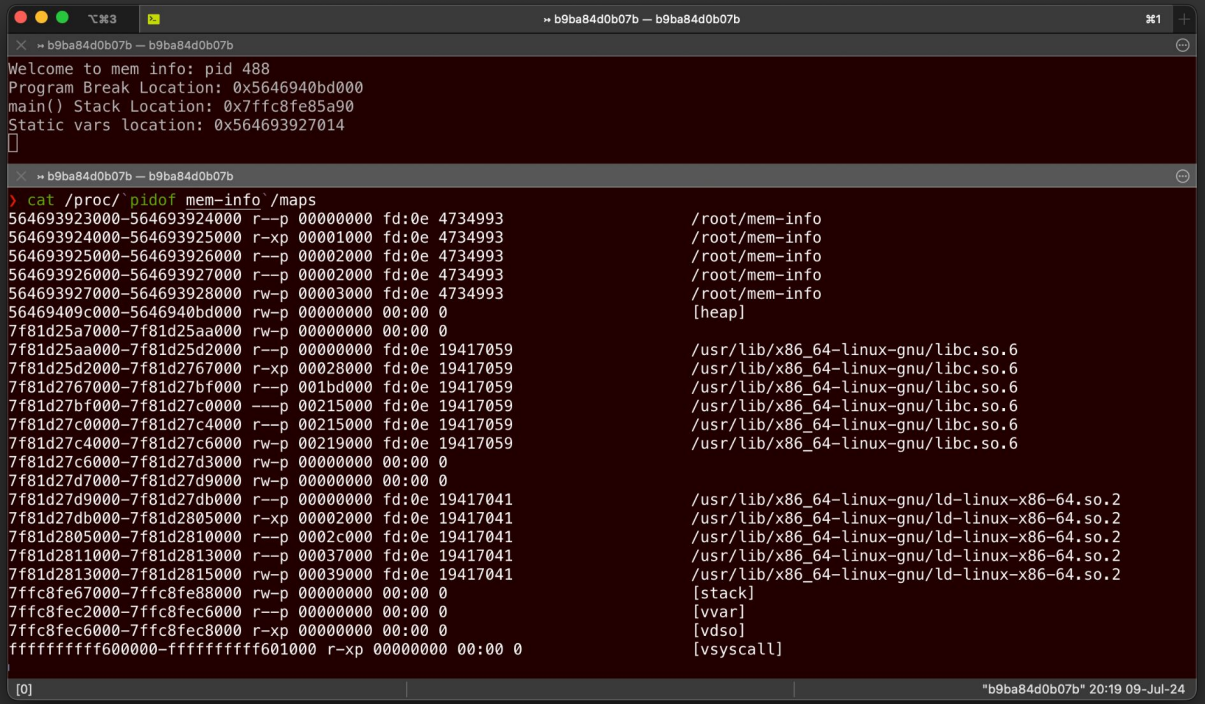
- VDSO variables



Process virtual memory layout (64 bits arm)



Process virtual memory layout (64 bits x86)



Malloc Internals (glibc) - basic concepts

Arena

- Memory management context (per process or per thread)
- Each arena has its heap(s) and bin structures
- Reduces contention in multithreaded programs

Heap

- Contiguous memory region (obtained from brk or mmap)
- Split into chunks for allocations (e.g., `malloc()`, `calloc()`, or `realloc()`)
- ***Ends with a top chunk***
- Each arena may manage multiple heaps

Bins

- Lists of freed chunks, organized by size
- ***Fast bins***: small, O(1) allocations
- Small/Large bins: exact or best-fit allocation
- Unsorted bin: recent frees before redistribution

```
Arena
 ├─ Heap #1 (brk / mmap)
 │     ├─ Chunks (allocated + free)
 │     └─ Top chunk
 ├─ Heap #2 (if needed)
 │     ├─ Chunks
 │     └─ Top chunk
 └─ Bins
       ├─ Fast bins   (tiny frees)
       ├─ Small bins  (exact size fit)
       ├─ Large bins  (best fit)
       └─ Unsorted bin (recent frees)
```

Malloc chunks

Malloc chunks

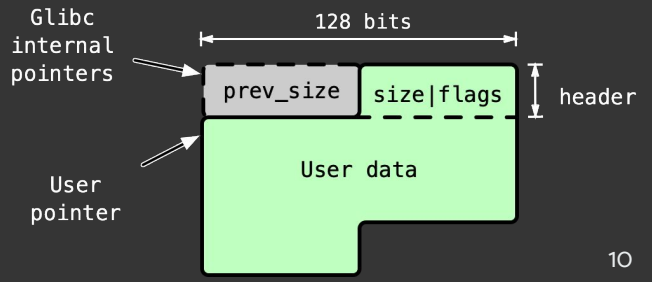
Memory is allocated and assigned to the user in chunks

- If X bytes is the amount of memory requested by the user, the amount effectively used by malloc is:

$$(X + H + (A - 1)) \;\&\; \sim (A - 1)$$

- On a 64 bit architecture:
  - H, header = 8 bytes for `size|flags`
    - `prev_size`, belongs to the prev_chunk
    - ***malloc internal pointers to a chunk point to prev_chunk*** (they consider the header 16 bytes long)
  - A, Alignment = 16 bytes; malloc alignment => **the 4 least significant bits of size are not used**
  - **Min size is 32 bytes =>** so `max(Y, 32)`, where Y is the formula above

- Flags - A,M,P: Least significant 3 bits of the header
  - P, the least significant bit, is PREV_INUSE (1 == in use)



Malloc chunks

Examples `malloc(X)`:

- `X=1  → max(( 1 + 8 + 15) & ~15, 32) = 32 bytes → usable 24`
- `X=16 → max((16 + 8 + 15) & ~15, 32) = 32 bytes → usable 24`
- `X=24 → max((24 + 8 + 15) & ~15, 32) = 32 bytes → usable 24`

- `X=25 → max((25 + 8 + 15) & ~15, 32) = 48 bytes → usable 40`
- `X=32 → max((32 + 8 + 15) & ~15, 32) = 48 bytes → usable 40`
- `X=40 → max((40 + 8 + 15) & ~15, 32) = 48 bytes → usable 40`

- `X=41 → max((41 + 8 + 15) & ~15, 32) = 64 bytes → usable 56`

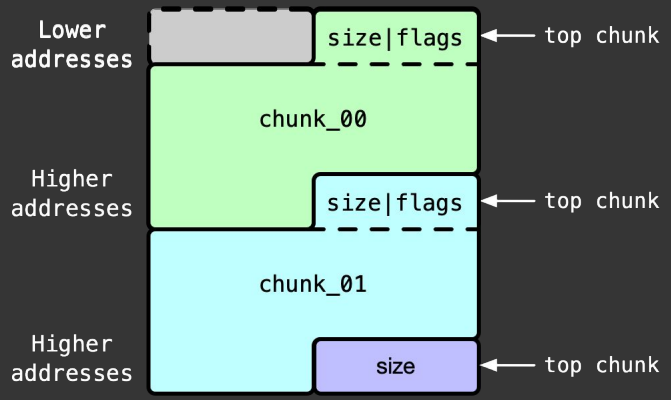
Top chunk

Glibc ***does not*** maintain per-allocation metadata beyond the chunk header

- It is up to the program to manage objects
- Higher-level languages may do that (e.g., using garbage collectors)

Glibc ***does*** track and manage available, including freed, memory

- More on the freed memory later
- *Normally*, malloc shrinks the so called **top chunk** to allocate memory to the user
  - **The top chunk** is a special malloc chunk: it only has a size
  - It is shrunk from the top (lower addresses) towards higher addresses



Example: allocation from top chunk

Top chunk

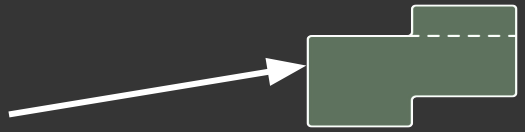
If the user requests more memory than is available in the top chunk (i.e., malloc size > top chunk size):

- malloc falls back to its slow path (sysmalloc) which tries, in order:
  - Extend the top chunk, or if that fails / the request is large,
  - Allocate via `mmap`, if that also fails
  - Return an error (NULL / errno)

Malloc chunks, hands on

`vis` command in pwndbg

Note: these slides often show chunks as blocks like this:



- That visualization helps reasoning on malloc heap exploits, however
- malloc-internal pointers to chunks ***account for the full header***: 0x10 bytes, i.e., 2 machine words

Exploitation Techniques

Malloc maleficarum

Malloc Maleficarum is a paper describing advanced heap exploitation techniques for glibc malloc

- House of Spirit
- ***House of Force***
- House of Orange
- House of Lore

And related, commonly used “non-house” primitives you’ll see alongside them:

- tcache poisoning / tcache stashing / ***fastbin dup*** / unsorted-bin leaks / fd/bk forging / FILE vtable abuse / ***hook overwrites***

Exploitation: House of Force

House of Force

House of Force exploits a **heap overflow** (**CWE-122**) into the top-chunk header, ultimately yielding an effective **write-what-where condition**

With the **House of Force**, we abuse the top chunk

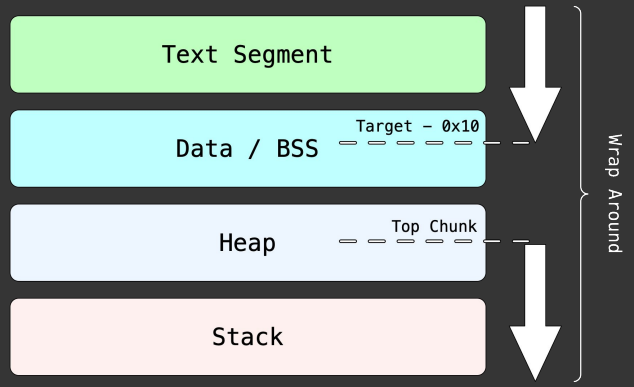
- The top chunk's size field says how much heap memory is available before needing more from the OS

House of Force - the high-level idea

If we can overwrite the size of the top chunk with a gigantic value (e.g., `0xffffffffffffffff`), we "trick" malloc into thinking it has that amount of space available...

- ...now you can request allocations of ***carefully chosen*** sizes that will move the top chunk to any address you want...
- ***Even wrapping around the available memory*** (using integer overflow)

giving you arbitrary reads and writes



Example: simplified process memory

House of Force

Simplest way to overwrite the size of the top chunk: heap based buffer overflow

House of Force

Step 1 - Overwrite the top chunk size, with a very large value

- A buffer overflow is ideal here (even an ***off-by-one vulnerability*** may be enough in some cases)

Step 2 - Request a large chunk to cover the distance from the current top chunk to target memory in Data/BSS

- To wrap around the process' virtual memory, the distance to target is:

```
MAX_UINT - (<heap base> + <bytes allocated so far>) + (<target> - <malloc header size>)
```

- Example, assume a `0x18` bytes chunk was allocated + `0x10` header (as seen by malloc):

```
distance_to_target = 0xFFFFFFFFFFFFFFFF - (heap_base + 0x28) + (elf.sym.target - 0x10)
```

Step 3 - Request a chunk to get access to the target memory

House of Force Demo

House of Force

House of Force only worked in older versions of Glibc

Nowadays we have strict checks on the top chunk size

- Since glibc 2.29+ there are checks in `malloc.c` that reject impossible top chunk sizes
- Overwriting the top chunk size with large values will cause an abort

```
malloc(): corrupted top size
```

Fastbins

Fastbins

Memory can be deallocated (freed) and reallocated multiple times during a program's life

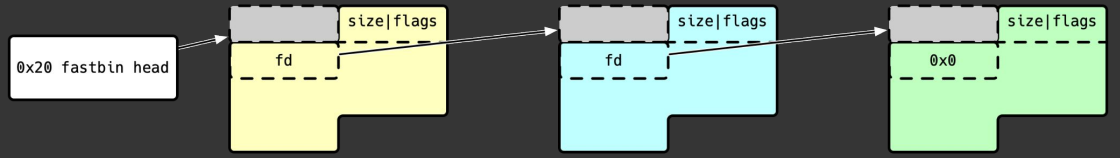
- Glibc implements optimizations to prevent memory fragmentation and allow faster operations, e.g.,
  - Freed memory might get merged back to the top chunk when possible and ***if over a certain size***
  - Freed memory gets organized in **bins of various sizes** (where chunks can be merged, sometimes)
  - Following memory requests, chunks can be **served from those bins** (even splitting them, sometimes)

So, the heap and its management can be quite complicated...

- For the sake of this introduction, we will only talk about the Fastbins

Keep in mind that after the introduction of tcache (recent versions of glibc), the behavior explained in the next slides changed

Fastbins



Singly-linked lists of freed chunks of sizes in the 0x20-0x80 bytes range (increments of 0x10 bytes)

- Each bin is the head of the list of chunks for a given size
- Fastbin (like other bins heads) are stored in malloc arenas (we will only use main arena here)
- When a 0x20-0x80 bytes chunk is free'd, it is linked in the corresponding fastbin
- The machine word after the `size|flag` field of each chunk is used as a forward pointer (fd)

Note: chunks of this size are not merged back with the top chunk(*), or with adjacent chunks when free'd

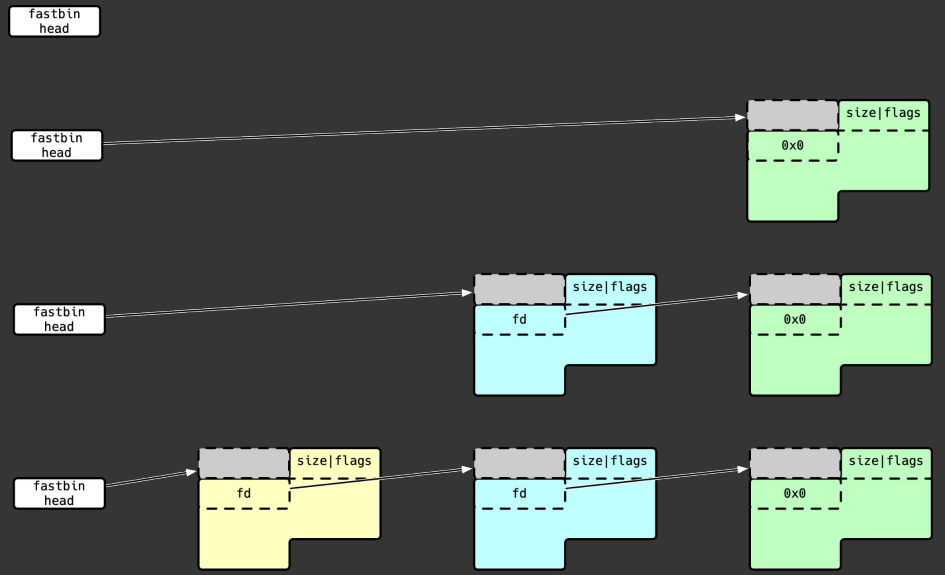
- Chunks larger than 0x80 bytes (after adding header/alignment) are merged back with the top chunk
- Example: `p = malloc(0x88)` followed by a `free(p)` returns the memory to the top chunk

(*) They can be merged later during a process called malloc_consolidate, which is triggered during larger malloc calls or other specific situations. This process flushes the fastbins into the unsorted bin, where they are coalesced.

Fastbins

On a `free()` for a 0x20-0x80 chunk

- Its `fd` pointer is set to the fastbin head value
- The fastbin head is set to the address of the chunk
  - This operation has O(1) complexity



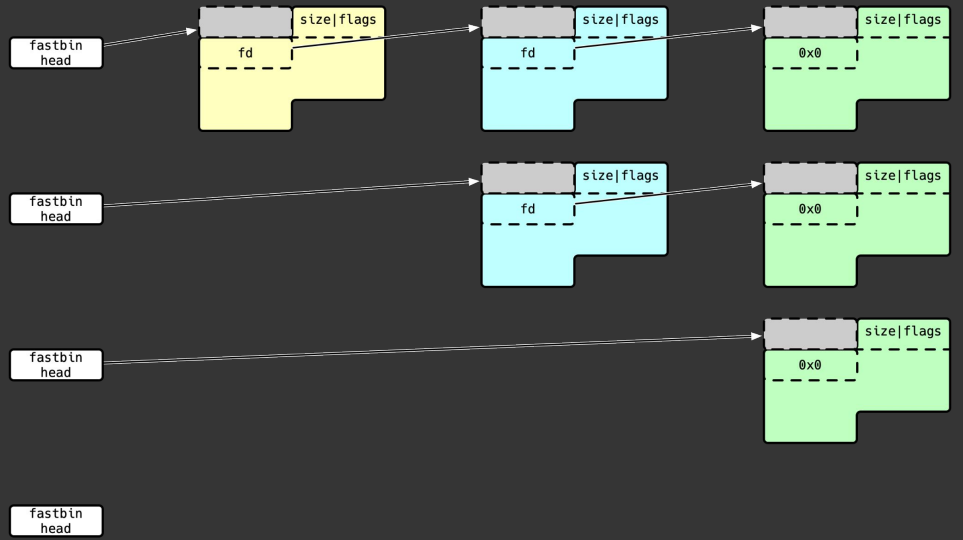
Example: release (free) of 3 chunks of the same size (top to bottom)

Fastbins

On a `malloc()` for 0x20-0x80 bytes, if the appropriate fastbin has chunks

- A chunk is served from the head of the list
- Its fd pointer is copied to the fastbin head
  - Again, O(1) complexity

Otherwise, memory is allocated from the top chunk



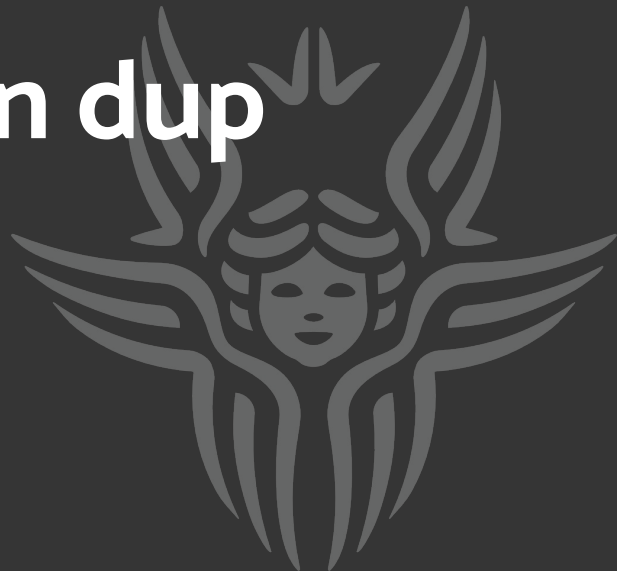
Example: allocation (malloc) of a chunk from a fastbin (top to bottom)

Fastbins - hands on

`fastbin` command in pwndbg

- The picture shows the heap content after we freed 3 chunks of size 0x20

Exploitation: Fastbin dup



Fastbin dup

Fastbin dup abuses a **double-free** (**CWE-415**) or fastbin fd corruption (e.g., CWE-787: Out-of-bounds Write), ultimately yielding a **write-what-where condition**

The high-level idea

- Double-free chunk X, reallocate the same size => we control X's content while it's still linked to the Fastbin list
- Manipulate the fd pointer to point to (semi-)arbitrary target memory
- Allocate from the Fastbin up until the target memory
- Now we have read/write access to target memory

Fastbin dup

Caveats

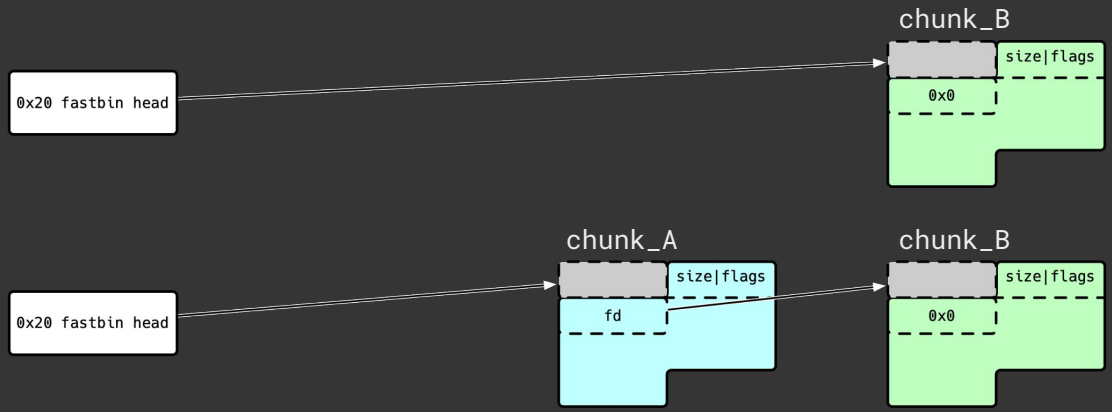
- `free()` detects double free if the double-freed chunk is linked to the head of the Fastbin
  - We cannot target the first linked chunk

- `malloc()` checks the size field in the header of chunks allocated from Fastbins
  - Target must ***resemble*** a valid chunk → size field needs to match the actual Fastbin size (e.g., `0x20` | `flags`, for a `0x20` Fastbin)
  - We can use **`find-fake-fast`** in pwndbg to find a "fake" chunk overlapping the target
  - ***Note:*** the fake chunk doesn't need to have any special memory alignment

Fastbin dup

Step 1 - Allocate 2 chunks within the Fastbin size range: **chunk_A** and **chunk_B**

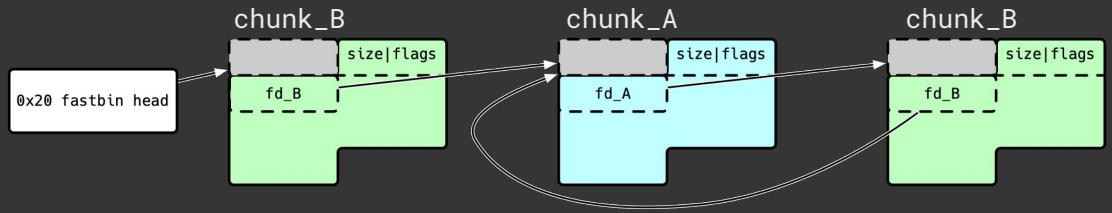
Step 2 - Free **chunk_B** then **chunk_A** ⇒ they will end in the corresponding Fastbin



Fastbin dup

Step 3 - Free **chunk_B** again

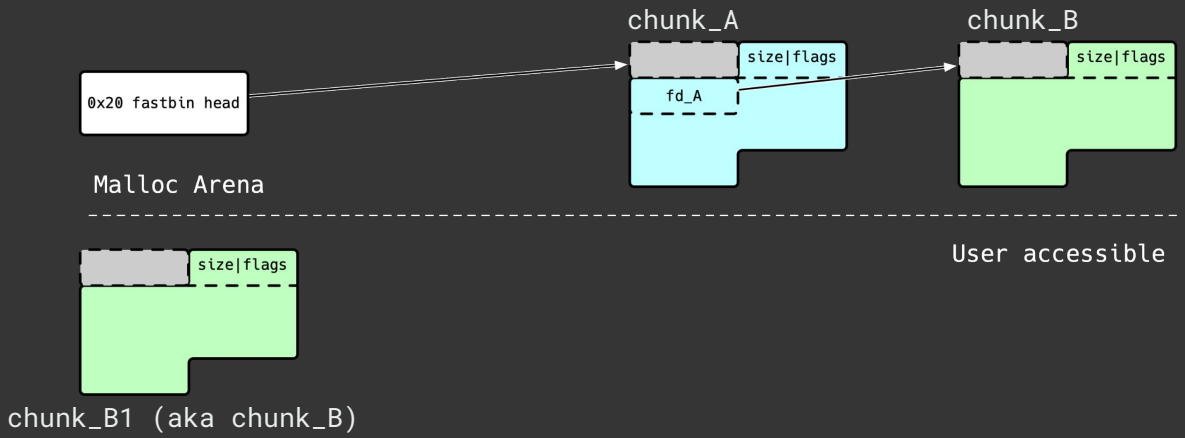
- glibc `free()` will add it again to the corresponding Fastbin, creating the situation depicted below
  - This way we won't trigger double free mitigation in free()



Fastbin dup

Step 4 - Request another chunk of the same size: **chunk_B1** (i.e., aka **chunk_B**)

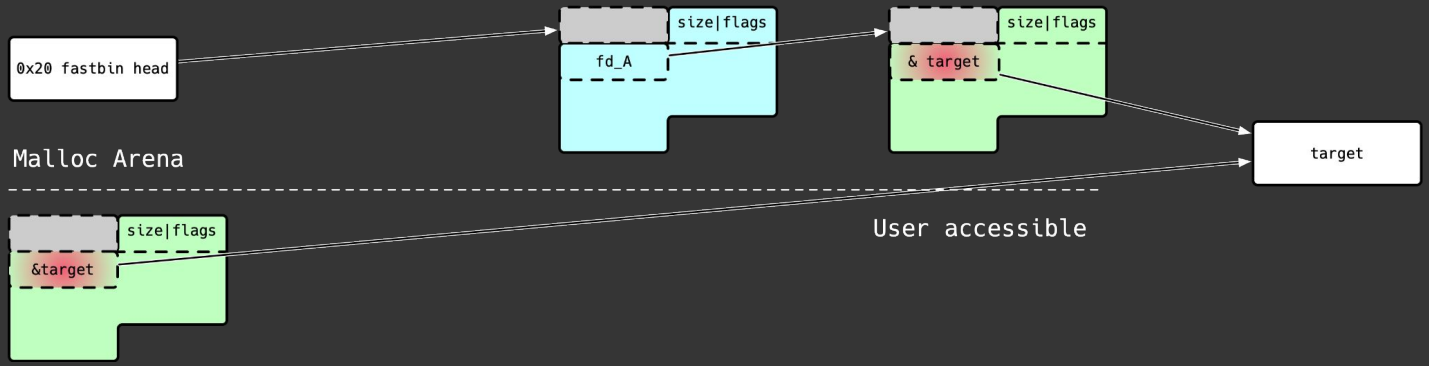
- We now have control over **chunk_B**, which is still linked in the Fastbin list



Fastbin dup

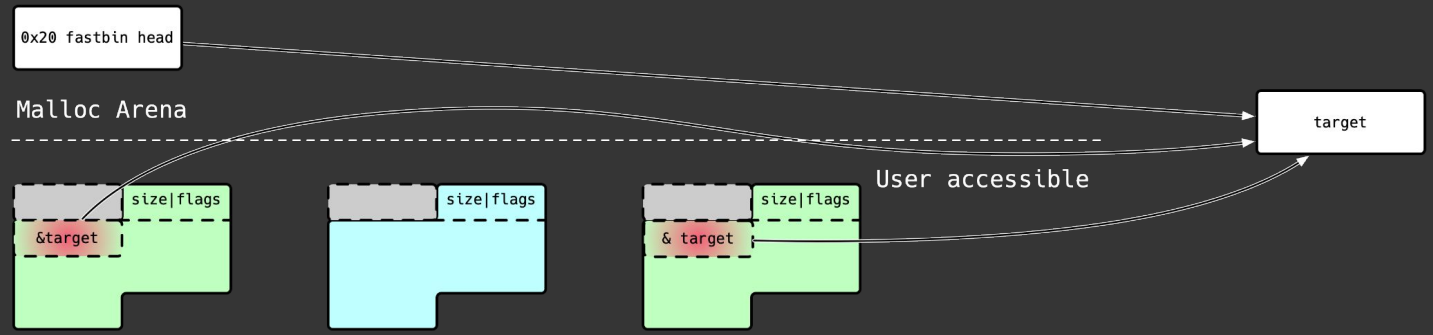
Step 5 - Overwrite **chunk_B**'s fd pointer with the target memory address

- **Remember**: the memory surrounding our target has some constraints:
  - The fake chunk size field is checked by `malloc()` - it must be the same as the Fastbin in use
  - On the bright side, the fake chunk doesn't need any special memory alignment



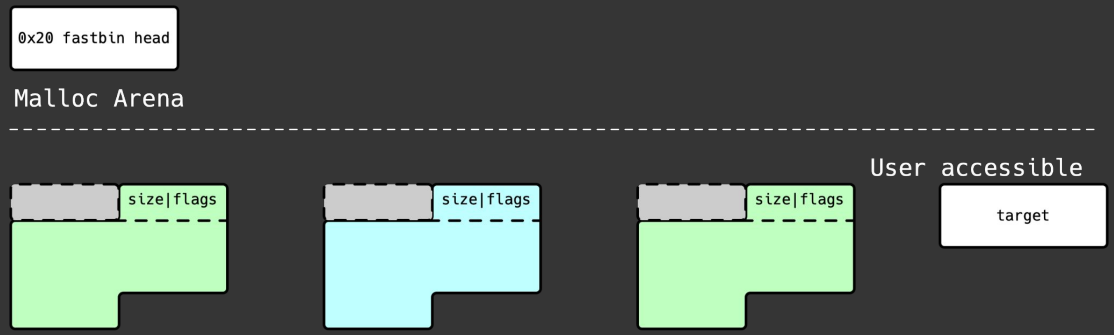
Fastbin dup

Step 6 - Allocate 2 more chunks from the same fastbin



Fastbin dup

Step 7 - Profit: an additional `malloc()` for the same fastbin will yield the target memory area



Fastbin dup Demo

From arbitrary writes to code execution

From arbitrary writes to code execution

Having a read/write primitive gives us some options for arbitrary code execution; however, remember:

- Limitations for the Fastbin dup technique (i.e., double-free check + chunk size must be in the Fastbin range)
- ASLR ← big deal

Typical techniques to achieve arbitrary code execution:

- Alter return addresses in the stack: return straight into some code or use ROP
- Alter the Global Offset Table (GOT), where not fully protected w/ RELRO
- Virtual tables / method tables
  - Leveraging structures used by higher level languages to implement virtual methods
- Allocator / teardown redirections
  - `__malloc_hook / __free_hook`
  - Corrupt teardown/exit vectors (e.g., destructor arrays, atexit lists) to run attacker-chosen code at exit

Malloc hooks

Allow intercepting memory allocation / deallocation; used for profiling, tracing, and debugging (e.g., valgrind)

- __malloc_hook ⟶ called ***instead*** of malloc(size) ; typically calls `malloc(size)` in turn
- __free_hook ⟶ called ***instead*** of free(ptr) ; typically calls `free(ptr)` in turn

Signature (`<malloc.h>`), example:

```
typeof(void *(size_t size, const void *caller)) *volatile  __malloc_hook;
typeof(void *(void *p, const void *caller)) *volatile  __free_hook;
```

Default value: NULL ⇒ no hook invoked; can be replaced at runtime

- If not NULL, the function pointed by the hook is called with the above params

Note: deprecated & removed after glibc 2.34 – replaced by malloc interposers (`LD_PRELOAD`)

Other useful concepts/tools

One gadget

- Cli tool to get a “magic” address in libc that directly executes a shell (e.g., /bin/sh) if execution is hijacked to it
    - Typically, a one gadget requires specific register or stack constraints (like `$rax == 0`) to be satisfied, so we need a bit of luck!

The role of ASLR

- Up to now, we had our demo program generously and intentionally leak addresses useful to determine glibc location in memory
- IRL we won’t have that luxury, so defeating ASLR becomes essential
- Sometimes `/proc/{pid}/maps` and `/proc/{pid}/map-files` (and `/proc/self`) may get in handy

HoF + malloc hooks C.E. - exploitation steps

Demo steps

- Step 1 - Corrupt Top Chunk Size
  - Allocate chunk_A and overflow it to overwrite the top_chunk's size with a massive value (`0xFF...FF`)
- Step 2 - Calculate Distance to the `__malloc_hook`
- Step 3 - Move Heap Top to Target
  - Allocate a new, very large chunk (chunk_B) using the distance calculated in step 2 (and accounting for a chunk header)
  - This chunk also conveniently stores the string "`/bin/sh\0`".
- Step 4 - Overwrite `__malloc_hook` with the address of glibc `system()`
- Step 5 - Trigger the Hook by calling malloc with the address of the chunk we allocated at step 3
  - Call malloc one last time to execute `system("/bin/sh\0")`

Fastbin dup + malloc hooks C.E. - exploitation steps

Demo steps

- Step 0 - Find a fake fastbin-sized chunk overlapping __malloc_hook - be that chunk_T
  - This will also tell us the chunk size(s) we need to use - be that 0x70 bytes
- Step 1 - Allocate 2 0x70 bytes chunks (0x68+0x8): chunk_A and chunk_B
- Step 2 - Free chunk_B then chunk_A ⇒ they will end in the corresponding Fastbin
- Step 3 - Free chunk_B again

  Step 4 - Request another 0x70 bytes chunk: `chunk_B1` (i.e., aka `chunk_B`)
  - We now have control over `chunk_B`, while it is still in the Fastbin list
- Step 5 - Overwrite chunk_B's fd pointer with the address of chunk_T
- Step 6 - Usign an appropriate padding, set the `__malloc_hook` to the address of a **one gadget** in glibc
- Step 7 - Allocate any memory size to trigger `__malloc_hook` and get a shell

Fastbin dup - code execution Demo

Links

- Malloc Maleficarum on Bugtraq - by Phantasmal Phantasmagoria, 2005
- Linux Heap Exploitation Part 1 on Udemy - by Max Camper
- Malloc hooks - man page
- CWE-122 - Heap buffer overflow
- CWE-415 - Double free